AuraMart

Phase 1 Build Guide

Foundation and local infrastructure — Sprint 0 and Sprint 1, every step end to end

Windows 11 · Spring Boot 4.1 · Java 21 · PostgreSQL 17 · GitHub Actions · written 16 Oct 2026

At a glance

Goal	The Catalog API runs on your local infrastructure, is tested in CI, documented, and released on GitHub as **v0.1.0**.
Covers	Sprint 0 (setup, 19–25 Oct 2026) and Sprint 1 (Catalog service, 26 Oct – 8 Nov 2026): stories AM-1 to AM-12, 27 story points.
Steps	19 steps. Each one has a goal, the exact commands or code, a check, and a commit.
Time	About 8 hours for Sprint 0 and 22.5 hours of building for Sprint 1, at 15 hours a week.
You need	A Windows 11 laptop and the **AuraMart** folder in your Downloads folder, so that you have **Downloads\AuraMart\START-HERE.txt**. Step 1 sets the laptop up.
Reference	**AuraMart\4-phase1-complete** is the finished project. Type the code yourself to learn; open that folder to compare a file, or when you are stuck.

What you will have at the end

- ☐ A GitHub repo with a protected main branch, pull requests and a green CI badge.
- ☐ PostgreSQL, Redis and Kafka running locally in Docker.
- ☐ A Catalog API with create, read, update, deactivate, paging, sorting and clean error responses.
- ☐ 13 automated tests that run on every push.
- ☐ Swagger UI, health checks, ADRs, user stories, a dev log and a v0.1.0 release.

> **Honest note**
>
> The code was written for Spring Boot 4.1 and checked for syntax, but it could not be compiled where it was written. Your first build in Step 2 and the test run in Step 15 are the real checks — if anything fails, Appendix D covers the likely causes.

Contents

Part A — What Phase 1 builds 3

Part B — Sprint 0: set everything up 5

Step 1 · Install and check your tools . . . 5
Step 2 · Create the local repository and build once . . . 5
Step 3 · Publish to GitHub and protect main . . . 6
Step 4 · Start the local infrastructure . . . 7
Step 5 · Turn on CI and make it required . . . 8
Step 6 · Your first pull request . . . 9
Step 7 · Set up the board and create the Sprint 1 issues . . . 10
Step 8 · Sprint 0 review and retrospective . . . 12

Part C — Sprint 1: build the Catalog service 13

Step 9 · Sprint 1 planning . . . 13
Step 10 · The story loop you will repeat . . . 14
Step 11 · Database schema and seed data with Flyway . . . 14
Step 12 · Create, update and deactivate products . . . 17
Step 13 · Browse with paging and sorting . . . 27
Step 14 · Price and stock status . . . 29
Step 15 · Automated tests . . . 30
Step 16 · Swagger UI and health checks . . . 36
Step 17 · Documentation that works on a fresh clone . . . 37

Part D — Finish Phase 1 38

Step 18 · Sprint review and release v0.1.0 . . . 38
Step 19 · Retrospective, velocity and what's next . . . 39

Appendix A. The project at the end of Phase 1 40

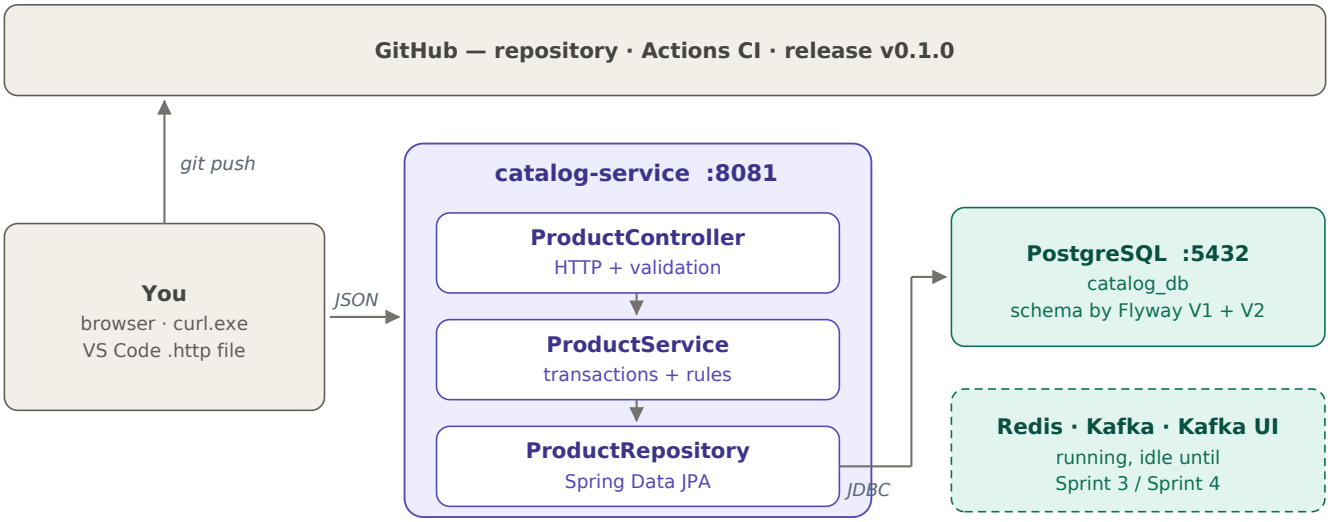
Appendix B. Build files 41

Appendix C. docker-compose.yml 43

Appendix D. Troubleshooting Phase 1 46

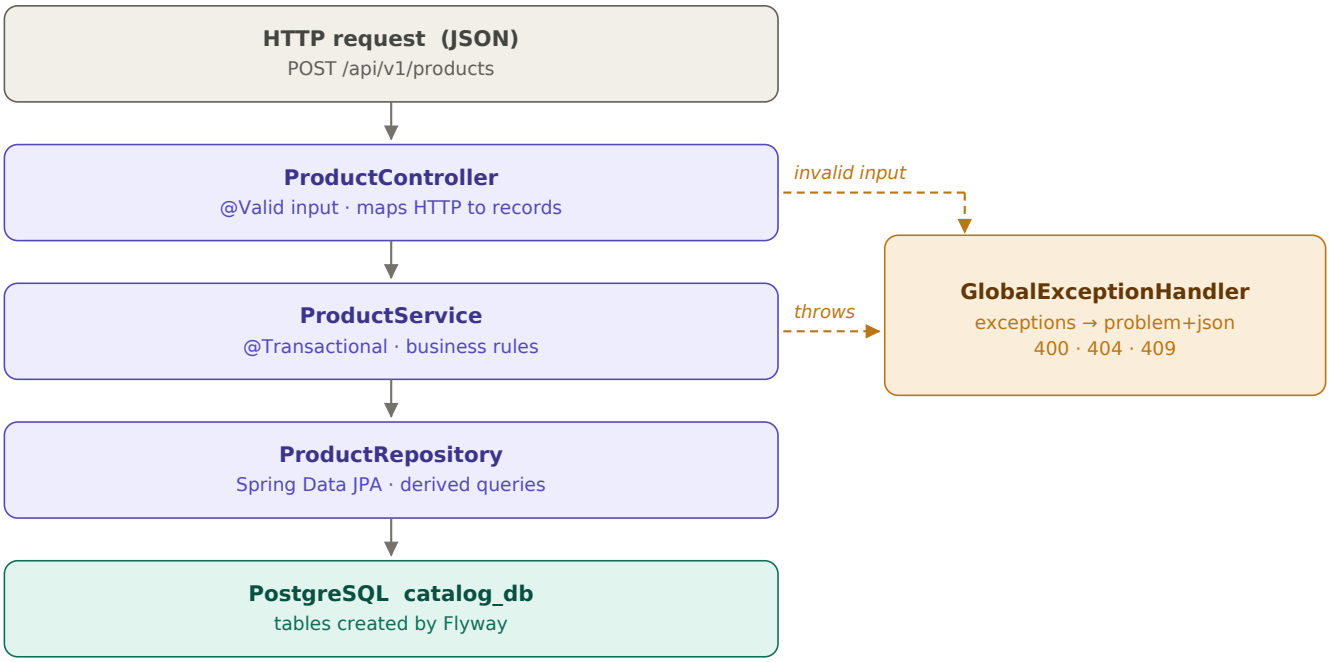
Part A — What Phase 1 builds

Phase 1 builds the first microservice, the Catalog service, and everything around it that a real team would set up first: a repository with rules, automated builds, local infrastructure and documentation. Redis and Kafka start now but stay idle until later sprints.



Inside the service: one request, layer by layer

Each layer has one job. The controller only speaks HTTP, the service owns transactions and rules, the repository talks to the database, and one exception handler turns every error into the same JSON shape.



What you work with: the AuraMart folder

1-docs	The four guides (this is number 4) and the feature list.
2-laptop-setup	Install and check scripts, docker-compose.yml, the CI workflow and config files. Step 1 runs the scripts; Step 2 copies the whole kit into your repo.
3-phase1-starter	The starting code. Step 2 copies it to **C:\dev\auramart**, where you do all your work.
4-phase1-complete	The finished Phase 1, for comparing. It has no Maven wrapper files: those are created on your laptop in Step 2.
5-frontend-prompt	A prompt for Claude Code that builds the complete web frontend on demo data — the Sprint 9–10 work, ready whenever you want it.

How every step is laid out

Part	What it tells you
Goal	What is true when the step is finished
Start	For stories: the branch to create, from an up-to-date main
Do	The exact commands or code, in order. Code listings are complete files
Check	How to prove it worked — the expected output
Ship it	Branch name and commit message; then the story loop from Step 10

Commands are for PowerShell in Windows Terminal, run from **C:\dev\auramart** unless a step says otherwise. Lines starting with # are comments. Every listing is labelled:

Label	What to do
from the starter	Already in your repo. Read it to understand it; nothing to change
from the kit	Came from 2-laptop-setup in Step 2; nothing to change
replace	Overwrite your file with this listing
new	Create the file with this content

Part B — Sprint 0: set everything up

One week, Monday 19 to Sunday 25 October 2026. No features yet: this week makes every later week faster.

Step 1 · Install and check your tools

AM-1 · Mon 19 Oct · about 2 hours

> **Goal**
>
> Java 21, Maven, Git, GitHub CLI, Docker Desktop, an IDE and Node.js are installed and verified.

Do

The **Laptop Setup Guide** (1-docs, number 3) explains every tool and has a manual path. The short version, from **Terminal (Admin)** (right-click the Start button):

```
# 1. Unzip AuraMart.zip into Downloads -> you get Downloads\AuraMart
Expand-Archive "$env:USERPROFILE\Downloads\AuraMart.zip" `
    -DestinationPath "$env:USERPROFILE\Downloads"

# 2. Install every tool (20-60 minutes, mostly downloading)
cd "$env:USERPROFILE\Downloads\AuraMart\2-laptop-setup"
powershell -ExecutionPolicy Bypass -File .\scripts\install-tools.ps1

# 3. Restart Windows, start Docker Desktop once, then install IntelliJ IDEA from JetBrains Toolbox
```

*Unzipping with File Explorer instead? **Extract All** suggests ...\Downloads\AuraMart — change it to ...\Downloads, or you get AuraMart inside AuraMart and the paths in this guide won't match.*

Check — in a new, normal terminal

```
java -version            # openjdk version "21.0.x" ... Temurin
mvn -v                   # Apache Maven 3.9.x ... Java version: 21.0.x
git --version            # git version 2.x
docker compose version   # Docker Compose version v2.x
gh auth status           # Logged in to github.com   (run "gh auth login" if not)
```

Or run `powershell -ExecutionPolicy Bypass -File .\scripts\check-env.ps1` from the 2-laptop-setup folder: every line under **Core tools (Sprint 0)** should say [OK].

Step 2 · Create the local repository and build once

AM-2 · Tue 20 Oct · about 1 hour

> **Goal**
>
> C:\dev\auramart holds the starter code plus the kit's files, is a Git repository with the Maven wrapper, and builds with all tests passing.

Do

```
# 1. Copy the starter code to C:\dev, then the kit's files on top of it
New-Item -ItemType Directory -Force -Path C:\dev | Out-Null
Copy-Item "$env:USERPROFILE\Downloads\AuraMart\3-phase1-starter\auramart" C:\dev -Recurse
Copy-Item "$env:USERPROFILE\Downloads\AuraMart\2-laptop-setup\*" C:\dev\auramart -Recurse -Force
cd C:\dev\auramart

# 2. Start Git and add the Maven wrapper (it pins the Maven version for everyone)
git init -b main
mvn -N wrapper:wrapper            # creates mvnw, mvnw.cmd and .mvn\wrapper\
```

```
.\mvnw.cmd -v                     # downloads Maven once, then prints the version

# 3. First build: compiles everything and runs the tests (downloads ~300 MB the first time)
.\mvnw.cmd -B verify
```

The second Copy-Item also replaces the starter's docker-compose.yml with the kit's version, which adds the opt-in profiles for later sprints.

Open the folder in IntelliJ IDEA (File → Open → C:\dev\auramart → Trust Project) and look around. The parent `pom.xml` manages every version (Appendix B). `.gitattributes` keeps line endings consistent between Windows and the Linux machines that run CI:

.gitattributes · from the kit

```
# Keep line endings consistent between Windows (your laptop) and Linux (CI, containers).
# Without this, ./mvnw breaks in GitHub Actions with
#   /usr/bin/env: 'sh\r': No such file or directory
* text=auto eol=lf

# Windows-only scripts keep Windows line endings
*.cmd text eol=crlf
*.bat text eol=crlf
*.ps1 text eol=crlf

# Binary files: never touch
*.png binary
*.jpg binary
*.gif binary
*.ico binary
*.jar binary
*.zip binary
*.pdf binary
```

Check

☐ The build ends with BUILD SUCCESS and “Tests run: 4, Failures: 0” (the starter's unit tests).

☐ mvnw, mvnw.cmd and the .mvn folder exist.

Commit (the very first one goes straight to main)

```
git add .
git update-index --chmod=+x mvnw      # lets ./mvnw run on Linux in CI
git commit -m "chore: initial scaffold with Maven wrapper (AM-2)"
```

Step 3 · Publish to GitHub and protect main

AM-2 · Tue 20 Oct · about 30 minutes

> **Goal**
>
> The repo is on GitHub, and nobody — including you — can push straight to main any more.

Do

```
gh repo create auramart --public --source=. --remote=origin --push
gh repo view --web       # opens the repo in your browser
```

Then protect main in the browser:

1. Repo → **Settings → Rules → Rulesets → New ruleset → New branch ruleset**.
2. Name: main. Enforcement status: **Active**. Target branches: **Add target → Include default branch**.

3. Tick **Require a pull request before merging**; set required approvals to **0** (GitHub doesn't let you approve your own pull request).
4. Make sure **Block force pushes** is ticked. Click **Create**. The CI check is added in Step 5, after its first run.

Check

☐ The repo page shows your files and the README.

☐ Settings → Rules → Rulesets shows “main” as Active.

Step 4 · Start the local infrastructure

AM-5 · Wed 21 Oct · about 1 hour

> **Goal**
>
> PostgreSQL (with pgvector), Redis, Kafka and Kafka UI run in Docker and pass their health checks.

Do

Make sure Docker Desktop says “Engine running”, then:

```
docker compose up -d      # first time: downloads about 1.2 GB of images
docker compose ps         # repeat until postgres, redis and kafka show (healthy)
```

The first start of PostgreSQL runs the init script below: one database and one login per service, plus the two extensions the Catalog service needs.

infra/postgres/init/01-create-service-databases.sql · from the starter

```
-- ================================================================
-- Runs once, when the postgres-data volume is empty.
-- To re-run it:  docker compose down -v && docker compose up -d
--
-- Database-per-service: each microservice owns its schema and data.
-- No service may read another service's tables — it must ask via API
-- or learn about changes via Kafka events. Separate logins enforce that.
--
-- Local-dev passwords only. Real secrets come from env vars / a vault.
-- ================================================================

-- Product Catalog (+ pgvector embeddings in Phase 4)
CREATE USER catalog_svc WITH PASSWORD 'catalog_pw';
CREATE DATABASE catalog_db OWNER catalog_svc;

-- Orders & checkout (+ transactional outbox in Phase 3)
CREATE USER order_svc WITH PASSWORD 'order_pw';
CREATE DATABASE order_db OWNER order_svc;

-- AI assistant (chat memory)
CREATE USER assistant_svc WITH PASSWORD 'assistant_pw';
CREATE DATABASE assistant_db OWNER assistant_svc;

-- Notification / fulfillment (processed-event log for idempotency)
CREATE USER notification_svc WITH PASSWORD 'notification_pw';
CREATE DATABASE notification_db OWNER notification_svc;

-- Extensions need superuser, so create them here rather than in Flyway.
\connect catalog_db
CREATE EXTENSION IF NOT EXISTS vector;     -- pgvector: semantic search
CREATE EXTENSION IF NOT EXISTS pg_trgm;    -- fuzzy keyword search fallback
```

Check

```
docker exec auramart-postgres psql -U catalog_svc -d catalog_db `
    -c "SELECT extname FROM pg_extension;"                      # -> plpgsql, vector, pg_trgm
docker exec auramart-redis redis-cli ping                       # -> PONG
docker exec auramart-kafka /opt/kafka/bin/kafka-topics.sh `
    --bootstrap-server localhost:9092 --list                    # -> empty, no error
```

Open http://localhost:8090 — Kafka UI shows the cluster “auramart-local” with 1 broker. The full Compose file is in Appendix C.

Step 5 · Turn on CI and make it required

AM-4 · Thu 22 Oct · about 30 minutes

> **Goal**
>
> Every push and pull request is built and tested on GitHub, and main only accepts green builds.

Do

The workflow file came with the kit and was in your first commit, so the push in Step 3 already ran it:

.github/workflows/ci.yml · from the kit

```
# Build and test every push to main and every pull request.
# Runs on Linux, so ./mvnw must have LF line endings and the execute bit (see the setup guide).
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v5

      - name: Set up JDK 21
        uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - name: Build and test
        run: ./mvnw -B verify
```

```
gh run list --workflow ci.yml --limit 3    # your first push's run: completed, success
```

If it failed, open it with `gh run view --web`*, fix the cause (Appendix D) and push again.*

Now require it: Settings → Rules → Rulesets → main → tick **Require status checks to pass** → **Add checks** → type **build** → select it → **Save changes**.

Check

☐ The Actions tab shows a green “CI” run for your first commit.

☐ The main ruleset lists “build” as a required status check.

Step 6 · Your first pull request

AM-2 · AM-4 · Thu 22 Oct · about 45 minutes

> **Goal**
>
> CONTRIBUTING.md, DEVLOG.md and the CI badge reach main through a pull request with a green build.

Do

```
git switch -c docs/AM-2-contributing
```

Create these two files in the repo root:

CONTRIBUTING.md · new

```
# Contributing to AuraMart

A solo project with team habits.

## Branches
- `main` is always green and releasable. Never commit to it directly.
- One branch per story: `feature/AM-<id>-<short-name>` (also `fix/`, `docs/`, `chore/`).

## Commit messages (Conventional Commits)
`<type>(<scope>): <summary> (AM-<id>)` - types: feat, fix, test, docs, refactor, chore, ci

Example: `feat(catalog): add product update endpoint (AM-7)`

## Pull requests
1. Push the branch and open a PR: `gh pr create --fill`
2. Wait for CI: `gh pr checks --watch`
3. Review your own diff with the checklist below - after a break, with fresh eyes
4. Squash merge: `gh pr merge --squash --delete-branch`

## Definition of Done
- [ ] Merged to main through a pull request, CI green
- [ ] Unit tests, plus an integration test when a database, cache or Kafka is involved
- [ ] Every acceptance criterion demonstrated
- [ ] API changes visible in Swagger; new configuration documented
- [ ] Docs updated (README, ADR, events.md) when something changed
- [ ] No secrets committed, no TODO without an issue
```

DEVLOG.md · new

```
# AuraMart dev log

Daily standup (2 minutes, at the start of every session) and sprint retrospectives.
Newest entry at the top.

## Template - daily standup
### <Day dd Mon yyyy> - Sprint <n> - Day <d>
Done yesterday:
Today:
Blockers:
Learned:

## Template - sprint retrospective
### Sprint <n> retrospective - <date>
```

DEVLOG.md · new

```
Energy (1-5):
What went well:
What didn't:
What I learned:
Last sprint's action done? yes / no
One action for next sprint (small, specific):
Velocity: planned <x> points, completed <y> points

---
```

In `README.md`, add the CI badge under the "# AuraMart" title line, with your GitHub username:

```
![CI](https://github.com/<your-github-username>/auramart/actions/workflows/ci.yml/badge.svg)
```

```
git add .
git commit -m "docs: contributing guide, dev log and CI badge (AM-2)"
git push -u origin HEAD
gh pr create --title "AM-2 Contributing guide, dev log and CI badge" --body "Sprint 0 setup"
gh pr checks --watch              # wait for "build" to pass
gh pr merge --squash --delete-branch
git switch main
git pull
```

Check

☐ The pull request shows a green "build" check and is merged.

☐ The README on GitHub shows a green "CI passing" badge.

Step 7 · Set up the board and create the Sprint 1 issues

AM-3 · Fri 23 Oct · about 1.5 hours

> **Goal**
>
> A GitHub Projects board shows Sprint 1's seven stories as issues, ready to plan.

Do — the board

1. GitHub → your profile → **Projects → New project**. Pick the **Kanban** template and name it "AuraMart". (With the plain Board template, edit the Status field instead.)
2. Status values, in order: **Backlog, Ready, In progress, In review, Done**.
3. Add fields (⋯ → Settings → New field): **Sprint** (Iteration, 2 weeks, starting 26 Oct 2026), **Points** (Number), **Epic** (Single select, E0–E8), **Priority** (Single select: Must, Should, Could).
4. ⋯ → Workflows: check that **Item closed** and **Pull request merged** set Status to Done (both are on by default).

Do — the issues

Start — move the card to In progress

```
git switch main
git pull
git switch -c chore/AM-3-issue-script
```

Create the script below (or copy it from 4-phase1-complete), then run it once:

scripts/create-sprint1-issues.ps1 · new

```
<#
```

scripts/create-sprint1-issues.ps1 · new

```
    AuraMart - create the Sprint 1 labels and issues on GitHub (story AM-3).

    Run from the repo folder, after "gh auth login":
        powershell -ExecutionPolicy Bypass -File .\scripts\create-sprint1-issues.ps1

    Running it twice creates the issues twice - run it once.
#>
$ErrorActionPreference = 'Continue'

if (-not (Get-Command gh -ErrorAction SilentlyContinue)) {
    Write-Host 'GitHub CLI (gh) not found. Install it and run "gh auth login" first.' `
        -ForegroundColor Red
    exit 1
}

# ------------------------------------------------------------------ labels
$labels = @(
    @{ Name = 'story';    Color = '534AB7'; Description = 'User story' },
    @{ Name = 'epic:E1';  Color = '0F6E56'; Description = 'E1 Product catalogue' },
    @{ Name = 'sprint:1'; Color = 'BA7517'; Description = 'Sprint 1 (26 Oct - 8 Nov 2026)' }
)
foreach ($label in $labels) {
    gh label create $label.Name --color $label.Color --description $label.Description --force |
        Out-Null
    Write-Host "label: $($label.Name)"
}

# ----------------------------------------------------------------- stories
$stories = @(
    @{ Id = 'AM-6'; Points = 2; Title = 'Flyway schema and seed data'
       Story = 'As a developer, I want the schema managed by Flyway with seed data so that ' +
               'every environment is identical.'
       Criteria = @('On a fresh volume, app start applies V1 and V2',
                    '12 seed products exist',
                    'Hibernate schema validation passes') },
    @{ Id = 'AM-7'; Points = 5; Title = 'Create, update and deactivate products'
       Story = 'As an admin, I want to create, update and deactivate products so that I can ' +
               'manage the catalogue.'
       Criteria = @('POST returns 201 and a Location header',
                    'PUT updates the product; the SKU never changes',
                    'DELETE returns 204 and the product is hidden',
                    'Duplicate SKU returns 409',
                    'Invalid body returns 400 problem+json with field errors') },
    @{ Id = 'AM-8'; Points = 3; Title = 'Browse products with paging and sorting'
       Story = 'As a shopper, I want to browse products by category with paging and sorting ' +
               'so that I find items quickly.'
       Criteria = @('?category=kitchen&sort=price,asc&size=5 returns correct page metadata',
                    'Inactive products are hidden',
                    'Page size is capped at 100') },
    @{ Id = 'AM-9'; Points = 2; Title = 'Show price and stock status'
       Story = 'As a shopper, I want to see price and stock status so that I know whether I ' +
               'can buy a product.'
       Criteria = @('GET /products/{id} returns price, currency, stockQuantity and inStock',
                    'Unknown id returns 404 problem+json') },
    @{ Id = 'AM-10'; Points = 3; Title = 'Automated tests for the catalog'
       Story = 'As a developer, I want automated tests for the catalog so that changes do not ' +
               'silently break it.'
       Criteria = @('Service unit tests: create, duplicate, not found, update',
                    'Controller tests for 201, 400, 404 and 409',
                    'All tests green in CI') },
    @{ Id = 'AM-11'; Points = 1; Title = 'Swagger UI and health endpoints'
       Story = 'As an API consumer, I want Swagger UI and health endpoints so that the API is ' +
               'discoverable and monitorable.'
       Criteria = @('/swagger-ui.html lists every endpoint with API title and version',
                    '/actuator/health shows UP including the database') },
```

scripts/create-sprint1-issues.ps1 · new

```
    @{ Id = 'AM-12'; Points = 1; Title = 'ADRs and verified run instructions'
       Story = 'As a reviewer, I want ADRs and verified run instructions so that decisions and ' +
               'setup are documented.'
       Criteria = @('ADR 0001-0003 in docs/adr with an index',
                    'README steps work exactly as written on a fresh clone') }
)

$tmp = Join-Path $env:TEMP 'auramart-issue-body.md'
foreach ($s in $stories) {
    $lines = @($s.Story, '', '**Acceptance criteria**')
    foreach ($c in $s.Criteria) { $lines += "- [ ] $c" }
    $lines += ''
    $lines += "Story points: $($s.Points)"
    [IO.File]::WriteAllText($tmp, ($lines -join "`n"))
    gh issue create --title "$($s.Id) $($s.Title)" --body-file $tmp `
        --label story --label 'epic:E1' --label 'sprint:1' | Out-Host
}
Remove-Item $tmp -ErrorAction SilentlyContinue

Write-Host ''
Write-Host 'Done. Add the issues to your project board: + Add item, then type # to pick them.' `
    -ForegroundColor Green
```

```
powershell -ExecutionPolicy Bypass -File .\scripts\create-sprint1-issues.ps1
gh issue list --label sprint:1        # seven issues, AM-6 to AM-12
```

On the board: **+ Add item**, type **#**, pick each issue; then set Sprint = Sprint 1, Points and Priority (they are in each issue's text). Then add Sprint 2's five stories (AM-13 to AM-17 in the Execution Plan) as draft items with their points; later stories can go in as one-line titles.

Ship it

Branch	`chore/AM-3-issue-script`
Commit	`chore: script that creates the Sprint 1 issues (AM-3)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 8 · Sprint 0 review and retrospective

Sat 24 Oct · about 45 minutes

> **Goal**
>
> You prove Sprint 0 is done, and decide one thing to do better next sprint.

Review — show that everything works

- ☐ docker compose ps: postgres, redis and kafka are (healthy).
- ☐ The Actions tab: every run green; the README badge says passing.
- ☐ The board: seven Sprint 1 stories in Backlog with points.
- ☐ Main is protected: a direct git push to main is rejected.
- ☐ The service starts: .\mvnw.cmd -pl services/catalog-service spring-boot:run logs “Started CatalogServiceApplication” (stop it with Ctrl+C).

Retrospective — add it to DEVLOG.md through a small pull request, as always

```
### Sprint 0 retrospective - Sat 24 Oct 2026
Energy (1-5): 4
What went well: all tools installed, first green CI run
What didn't: Docker Desktop needed two restarts
What I learned: why .gitattributes matters for mvnw in Linux CI
One action for next sprint: commit after every small working change
Velocity: planned 10 points, completed 10 points
```

The lines above are an example — write your own answers.

Part C — Sprint 1: build the Catalog service

Two weeks, Monday 26 October to Sunday 8 November 2026. Seven stories, 17 points, 22.5 hours planned out of 27 available.

Step 9 · Sprint 1 planning

Mon 26 Oct · 1 hour

> **Goal**
>
> A written sprint goal, the seven stories committed, and each story split into small tasks.

> **Sprint goal**
>
> **The Catalog API runs end to end on local infrastructure, is tested in CI and documented.**

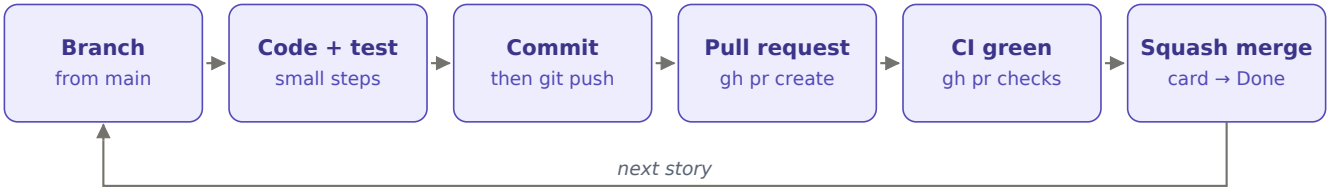
Story	Pts	Hours	Step
AM-6 Flyway schema and seed data	2	2.5	11
AM-7 Create, update and deactivate products	5	7.5	12
AM-8 Browse with paging and sorting	3	3	13
AM-9 Price and stock status	2	2	14
AM-10 Automated tests	3	5	15
AM-11 Swagger UI and health	1	1	16
AM-12 ADRs and run instructions	1	1.5	17
Total	17	22.5	

On the board, move all seven to **Ready**. The task-by-task breakdown is in the Execution Plan (1-docs, number 2), Section 7.

Step 10 · The story loop you will repeat

every story

Every Sprint 1 story follows the same six moves. Learn them once:




```
# 1. Start the story from an up-to-date main; move its card to In progress
git switch main
git pull
git switch -c feature/AM-7-product-crud

# 2. Work in small steps; commit whenever something works
git add .
git commit -m "feat(catalog): product CRUD with problem+json errors (AM-7)"

# 3. Publish and open the pull request ("Closes" links it to the issue); card to In review
git push -u origin HEAD
gh issue list --label sprint:1                        # find the issue number
gh pr create --title "AM-7 Product CRUD" --body "Closes #<issue-number>"

# 4. Wait for CI, read your own diff like a reviewer, merge (the card moves to Done)
gh pr checks --watch
gh pr diff
gh pr merge --squash --delete-branch

# 5. Back to main for the next story
git switch main
git pull
```

- Each working day, add a three-line standup to DEVLOG.md: done yesterday, doing today, blockers.
- Restart the service after every code change: Ctrl+C in its terminal, then the spring-boot:run command again.

Step 11 · Database schema and seed data with Flyway

AM-6 · 2 points · about 2.5 hours

> **Goal**
>
> On an empty database, starting the service creates the products table and 12 demo products, and Hibernate confirms the Java classes match the table.

Start — move the card to In progress

```
git switch main
git pull
git switch -c feature/AM-6-flyway
```

Flyway runs every file in db/migration that it hasn't run yet, in version order, and records each one in the `flyway_schema_history` table. Both migrations are already in your repo — read them first:

services/catalog-service/src/main/resources/db/migration/V1__create_products.sql · from the starter

```
-- Products owned by the catalog service.
-- stock_quantity lives here (not a separate inventory service) to keep the
```

services/catalog-service/src/main/resources/db/migration/V1__create_products.sql · from the starter

```
-- service count manageable for a solo developer. In Phase 3 the order saga
-- reserves stock by sending a command to this service over Kafka.

CREATE TABLE products (
    id              UUID            PRIMARY KEY,
    sku             VARCHAR(64)     NOT NULL,
    name            VARCHAR(200)    NOT NULL,
    description     TEXT            NOT NULL,
    category        VARCHAR(80)     NOT NULL,
    brand           VARCHAR(80),
    price           NUMERIC(12, 2)  NOT NULL CHECK (price >= 0),
    currency        VARCHAR(3)      NOT NULL DEFAULT 'INR',   -- ISO 4217
    stock_quantity  INTEGER         NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    active          BOOLEAN         NOT NULL DEFAULT TRUE,
    version         BIGINT          NOT NULL DEFAULT 0,   -- optimistic locking
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uk_products_sku UNIQUE (sku)
);

CREATE INDEX idx_products_category_active ON products (category) WHERE active;

-- Trigram index: cheap fuzzy keyword search now, and the "keyword" half of
-- hybrid (keyword + vector) search in Phase 4.
CREATE INDEX idx_products_name_trgm ON products USING gin (name gin_trgm_ops);
```

services/catalog-service/src/main/resources/db/migration/V2__seed_demo_products.sql · from the starter

```
-- Demo catalogue. Descriptions are deliberately rich: in Phase 4 they become the text
-- that gets embedded for semantic search ("something to keep my tea hot on a long
-- train journey" should find the steel flask). Long texts are joined with ||.

INSERT INTO products (id, sku, name, description, category, brand, price, stock_quantity)
VALUES
('0b9a7c1e-0001-4c3a-9a10-000000000001', 'AUD-ANC-100', 'Aura Pods ANC Earbuds',
 'True wireless earbuds with active noise cancellation, 30-hour battery with case, '
 || 'IPX5 sweat resistance and low-latency gaming mode. Good for commutes and the gym.',
 'electronics', 'AuraSound', 4999.00, 120),

('0b9a7c1e-0002-4c3a-9a10-000000000002', 'AUD-OVR-200', 'Studio Over-Ear Headphones',
 'Closed-back over-ear headphones with memory-foam cushions, 40mm drivers and a '
 || 'detachable cable. Comfortable for long work-from-home calls and music production.',
 'electronics', 'AuraSound', 8999.00, 45),

('0b9a7c1e-0003-4c3a-9a10-000000000003', 'KIT-FLK-750', 'Steel Vacuum Flask 750ml',
 'Double-wall stainless steel flask that keeps tea or coffee hot for 18 hours and drinks '
 || 'cold for 24. Leak-proof lid, fits car cup holders. Ideal for travel and office.',
 'kitchen', 'Thermique', 1299.00, 300),

('0b9a7c1e-0004-4c3a-9a10-000000000004', 'KIT-PRS-500', 'Pressure Cooker 5L Induction',
 'Five-litre stainless steel pressure cooker with an induction-compatible base, safety '
 || 'valve and gasket release. Cooks dal and rice for a family of four.',
 'kitchen', 'HomeChef', 2499.00, 80),

('0b9a7c1e-0005-4c3a-9a10-000000000005', 'FIT-YMT-006', 'Anti-Slip Yoga Mat 6mm',
 'Six-millimetre TPE yoga mat with an anti-slip texture on both sides and a carry strap. '
 || 'Cushions knees on hard floors; lightweight enough to take to class.',
 'fitness', 'FlexForm', 899.00, 200),

('0b9a7c1e-0006-4c3a-9a10-000000000006', 'FIT-DMB-010', 'Adjustable Dumbbell Set 10kg',
 'Pair of adjustable dumbbells from 2.5kg to 10kg each with spin-lock collars. A compact '
 || 'home-gym option for strength training in a small apartment.',
 'fitness', 'FlexForm', 3499.00, 60),
```

services/catalog-service/src/main/resources/db/migration/V2__seed_demo_products.sql · from the starter

```
('0b9a7c1e-0007-4c3a-9a10-000000000007', 'BAG-LPT-015', 'Laptop Backpack 15.6"',
 'Water-resistant backpack with a padded 15.6-inch laptop sleeve, USB charging port, '
 || 'hidden anti-theft pocket and luggage strap. Built for daily office commutes and '
 || 'short trips.',
 'bags', 'UrbanTrail', 1799.00, 150),

('0b9a7c1e-0008-4c3a-9a10-000000000008', 'BAG-TRL-055', 'Cabin Trolley Bag 55cm',
 'Hard-shell polycarbonate cabin luggage with 360-degree spinner wheels and a TSA lock. '
 || 'Meets most domestic airline cabin size limits.',
 'bags', 'UrbanTrail', 4299.00, 40),

('0b9a7c1e-0009-4c3a-9a10-000000000009', 'HOM-LMP-LED', 'Smart LED Desk Lamp',
 'Dimmable LED desk lamp with five colour temperatures, eye-care flicker-free light and '
 || 'a USB-C port. Ideal for late-night coding or study sessions.',
 'home', 'Lumina', 1599.00, 90),

('0b9a7c1e-0010-4c3a-9a10-000000000010', 'HOM-PUR-HEP', 'HEPA Air Purifier',
 'True HEPA H13 air purifier covering rooms up to 350 sq ft. Removes PM2.5, dust and '
 || 'pollen, with a quiet night mode. Helpful during high-pollution winter months.',
 'home', 'PureNest', 9999.00, 25),

('0b9a7c1e-0011-4c3a-9a10-000000000011', 'BOK-JAV-021', 'Modern Java in Action (Paperback)',
 'Practical guide to modern Java: records, streams, virtual threads and pattern '
 || 'matching, with worked examples for backend developers.',
 'books', 'TechPress', 699.00, 75),

-- Out of stock on purpose: lets you test inStock = false (and, later, the AI's answer)
('0b9a7c1e-0012-4c3a-9a10-000000000012', 'ELE-PWB-20K', 'Power Bank 20000mAh',
 '20000mAh power bank with 22.5W fast charging, USB-C power delivery and two USB-A '
 || 'ports. Charges a phone four times over; airline carry-on friendly.',
 'electronics', 'VoltX', 1899.00, 0);
```

> **Golden rule of migrations**
>
> Never edit a migration that has already run — add a V3 instead. Flyway stores a checksum of every file it ran and stops with “Validate failed: Migration checksum mismatch” if one changes.

Do — the fresh-database test

```
docker compose down -v      # wipe local data, so Flyway starts from an empty database
docker compose up -d
docker compose ps           # wait until postgres shows (healthy)
.\mvnw.cmd -pl services/catalog-service spring-boot:run
```

Check — in the log, then in a second terminal

```
# Log lines to look for:
#   Successfully applied 2 migrations to schema "public", now at version v2
#   Started CatalogServiceApplication in ... seconds

docker exec auramart-postgres psql -U catalog_svc -d catalog_db `
    -c "SELECT version, description, success FROM flyway_schema_history;"     # 2 rows, t
docker exec auramart-postgres psql -U catalog_svc -d catalog_db `
    -c "SELECT count(*) FROM products;"                                       # 12
```

No “Schema-validation” error at startup means Hibernate's check passed (ddl-auto: validate). Write what you saw in DEVLOG.md — that note is this story's commit.

Ship it

Branch	`feature/AM-6-flyway`
Commit	`docs(catalog): verify Flyway migrations on a fresh database (AM-6)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 12 · Create, update and deactivate products

AM-7 · 5 points · about 7.5 hours

> **Goal**
>
> POST, PUT and DELETE work, with 201/200/204 on success and clear problem+json errors: 400 for invalid input, 404 for unknown products, 409 for duplicate SKUs.

Start — move the card to In progress

```
git switch main
git pull
git switch -c feature/AM-7-product-crud
```

The starter already has a first version of the CRUD code. This story reads it end to end, makes two real changes — the entity and the error handler — and proves every acceptance criterion.

The entity — replace

What changes: the SKU column becomes **updatable = false**, so not even a bug in the service can change a SKU, and the timestamps use standard JPA lifecycle callbacks (@PrePersist, @PreUpdate) with the exact column type that V1 created.

services/catalog-service/src/main/java/com/auramart/catalog/product/Product.java · replace

```
package com.auramart.catalog.product;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.PrePersist;
import jakarta.persistence.PreUpdate;
import jakarta.persistence.Table;
import jakarta.persistence.Version;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

/**
 * Product aggregate - the persistence model. It never leaves the service: controllers
 * speak in ProductResponse records, so the table can change without breaking API clients.
 *
 * Behaviour (stock changes, deactivation; stock reservation in Phase 3) lives here, not in
 * a service full of setters, so the rules are enforced in exactly one place.
 */
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
```

services/catalog-service/src/main/java/com/auramart/catalog/product/Product.java · replace

```
    /** Permanent business identifier: updatable = false means Hibernate never changes it. */
    @Column(nullable = false, unique = true, length = 64, updatable = false)
    private String sku;

    @Column(nullable = false, length = 200)
    private String name;

    @Column(nullable = false)
    private String description;

    @Column(nullable = false, length = 80)
    private String category;

    @Column(length = 80)
    private String brand;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal price;

    @Column(nullable = false, length = 3)
    private String currency;

    @Column(name = "stock_quantity", nullable = false)
    private int stockQuantity;

    @Column(nullable = false)
    private boolean active = true;

    /** Optimistic locking: two concurrent updates can't both win (Phase 3 relies on it). */
    @Version
    private long version;

    // columnDefinition names the exact type Flyway created (V1), so Hibernate's
    // "ddl-auto: validate" check compares like with like.
    @Column(name = "created_at", nullable = false, updatable = false,
            columnDefinition = "timestamptz")
    private Instant createdAt;

    @Column(name = "updated_at", nullable = false, columnDefinition = "timestamptz")
    private Instant updatedAt;

    protected Product() {
        // required by JPA
    }

    public Product(String sku, String name, String description, String category,
                   String brand, BigDecimal price, String currency, int stockQuantity) {
        this.sku = sku;
        applyDetails(name, description, category, brand, price, currency);
        setStock(stockQuantity);
    }

    public void applyDetails(String name, String description, String category,
                             String brand, BigDecimal price, String currency) {
        this.name = name;
        this.description = description;
        this.category = category.toLowerCase();
        this.brand = brand;
        this.price = price;
        this.currency = currency.toUpperCase();
    }

    public void setStock(int quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException("Stock cannot be negative");
```

services/catalog-service/src/main/java/com/auramart/catalog/product/Product.java · replace

```
        }
        this.stockQuantity = quantity;
    }

    public void deactivate() {
        this.active = false;
    }

    public boolean isInStock() {
        return stockQuantity > 0;
    }

    // Standard JPA lifecycle callbacks keep the timestamps right on every insert and update.
    @PrePersist
    void onCreate() {
        Instant now = Instant.now();
        this.createdAt = now;
        this.updatedAt = now;
    }

    @PreUpdate
    void onUpdate() {
        this.updatedAt = Instant.now();
    }

    public UUID getId() { return id; }
    public String getSku() { return sku; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public String getCategory() { return category; }
    public String getBrand() { return brand; }
    public BigDecimal getPrice() { return price; }
    public String getCurrency() { return currency; }
    public int getStockQuantity() { return stockQuantity; }
    public boolean isActive() { return active; }
    public long getVersion() { return version; }
    public Instant getCreatedAt() { return createdAt; }
    public Instant getUpdatedAt() { return updatedAt; }
}
```

Request and response records

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductRequest.java · from the starter

```
package com.auramart.catalog.product;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;

import java.math.BigDecimal;

/**
 * Inbound payload for create and update. A record is immutable, has no boilerplate, and
 * the validation annotations sit right on its components.
 * On update the SKU is ignored: SKUs are permanent identifiers.
 */
public record ProductRequest(

        @NotBlank
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductRequest.java · from the starter

```
        @Size(max = 64)
        @Pattern(regexp = "[A-Z0-9-]+", message = "use upper-case letters, digits and dashes")
        String sku,

        @NotBlank @Size(max = 200) String name,
        @NotBlank @Size(max = 4000) String description,
        @NotBlank @Size(max = 80) String category,
        @Size(max = 80) String brand,

        @NotNull @DecimalMin("0.00") @Digits(integer = 10, fraction = 2)
        BigDecimal price,

        @NotBlank
        @Pattern(regexp = "[A-Za-z]{3}", message = "must be a 3-letter ISO currency code")
        String currency,

        @PositiveOrZero int stockQuantity
) {
}
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductResponse.java · from the starter

```
package com.auramart.catalog.product;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

/** Outbound API shape. Deliberately hides `version` and `active`. */
public record ProductResponse(
        UUID id,
        String sku,
        String name,
        String description,
        String category,
        String brand,
        BigDecimal price,
        String currency,
        int stockQuantity,
        boolean inStock,
        Instant updatedAt
) {
    static ProductResponse from(Product p) {
        return new ProductResponse(
                p.getId(), p.getSku(), p.getName(), p.getDescription(), p.getCategory(),
                p.getBrand(), p.getPrice(), p.getCurrency(), p.getStockQuantity(),
                p.isInStock(), p.getUpdatedAt());
    }
}
```

Repository and exceptions

Spring Data writes the SQL from the method names: findByActiveTrueAndCategory becomes WHERE active = true AND category = ?.

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductRepository.java · from the starter

```
package com.auramart.catalog.product;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductRepository.java · from the starter

```
import java.util.UUID;

public interface ProductRepository extends JpaRepository<Product, UUID> {

    Page<Product> findByActiveTrue(Pageable pageable);

    Page<Product> findByActiveTrueAndCategory(String category, Pageable pageable);

    Optional<Product> findByIdAndActiveTrue(UUID id);

    boolean existsBySku(String sku);
}
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductExceptions.java · from the starter

```
package com.auramart.catalog.product;

import java.util.UUID;

/** Domain exceptions — mapped to HTTP status codes in GlobalExceptionHandler. */
public final class ProductExceptions {

    private ProductExceptions() {
    }

    public static class ProductNotFoundException extends RuntimeException {
        public ProductNotFoundException(UUID id) {
            super("Product " + id + " not found");
        }
    }

    public static class DuplicateSkuException extends RuntimeException {
        public DuplicateSkuException(String sku) {
            super("A product with SKU " + sku + " already exists");
        }
    }
}
```

Service

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductService.java · from the starter

```
package com.auramart.catalog.product;

import com.auramart.catalog.product.ProductExceptions.DuplicateSkuException;
import com.auramart.catalog.product.ProductExceptions.ProductNotFoundException;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.UUID;

/**
 * Application service: transaction boundaries and orchestration.
 * Reads are readOnly, so Hibernate skips dirty-checking (and could use a read replica later).
 *
 * Phase 2 adds @Cacheable / @CacheEvict (Redis) here.
 * Phase 4 re-embeds a product for semantic search when it is created or updated.
 */
@Service
@Transactional(readOnly = true)
public class ProductService {
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductService.java · from the starter

```
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public Page<ProductResponse> list(String category, Pageable pageable) {
        Page<Product> page = (category == null || category.isBlank())
                ? repository.findByActiveTrue(pageable)
                : repository.findByActiveTrueAndCategory(category.toLowerCase(), pageable);
        return page.map(ProductResponse::from);
    }

    public ProductResponse get(UUID id) {
        return ProductResponse.from(findActive(id));
    }

    @Transactional
    public ProductResponse create(ProductRequest request) {
        if (repository.existsBySku(request.sku())) {
            throw new DuplicateSkuException(request.sku());
        }
        Product product = new Product(
                request.sku(), request.name(), request.description(), request.category(),
                request.brand(), request.price(), request.currency(), request.stockQuantity());
        return ProductResponse.from(repository.save(product));
    }

    @Transactional
    public ProductResponse update(UUID id, ProductRequest request) {
        Product product = findActive(id);
        product.applyDetails(request.name(), request.description(), request.category(),
                request.brand(), request.price(), request.currency());
        product.setStock(request.stockQuantity());
        // The entity is managed, so Hibernate would save the changes at commit anyway.
        // saveAndFlush writes them now, so the response already shows the new updatedAt.
        return ProductResponse.from(repository.saveAndFlush(product));
    }

    /** Soft delete: past orders still point at this product. */
    @Transactional
    public void deactivate(UUID id) {
        findActive(id).deactivate();
    }

    private Product findActive(UUID id) {
        return repository.findByIdAndActiveTrue(id)
                .orElseThrow(() -> new ProductNotFoundException(id));
    }
}
```

Controller

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductController.java · from the starter

```
package com.auramart.catalog.product;

import jakarta.validation.Valid;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.web.PageableDefault;
import org.springframework.data.web.PagedModel;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductController.java · from the starter

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.UUID;

/**
 * Thin HTTP adapter: parse, validate, delegate, shape the response. No business logic.
 * The /api/v1 prefix lets a breaking change ship later as /api/v2 alongside it.
 */
@RestController
@RequestMapping("/api/v1/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    /**
     * {@code GET /api/v1/products?category=kitchen&page=0&size=20&sort=price,asc}
     * PagedModel gives a stable JSON shape (content + page metadata) instead of
     * serialising Spring's internal PageImpl class.
     */
    @GetMapping
    public PagedModel<ProductResponse> list(
            @RequestParam(required = false) String category,
            @PageableDefault(size = 20, sort = "name", direction = Sort.Direction.ASC)
            Pageable pageable) {
        return new PagedModel<>(service.list(category, pageable));
    }

    @GetMapping("/{id}")
    public ProductResponse get(@PathVariable UUID id) {
        return service.get(id);
    }

    @PostMapping
    public ResponseEntity<ProductResponse> create(@Valid @RequestBody ProductRequest request) {
        ProductResponse created = service.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }

    @PutMapping("/{id}")
    public ProductResponse update(@PathVariable UUID id,
                                  @Valid @RequestBody ProductRequest request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deactivate(@PathVariable UUID id) {
        service.deactivate(id);
        return ResponseEntity.noContent().build();
    }
```

services/catalog-service/src/main/java/com/auramart/catalog/product/ProductController.java · from the starter

```
}
```

Error handling — replace

What changes: validation errors now list every bad field in an **errors** map, and a database constraint violation becomes a 409 instead of a 500.

services/catalog-service/src/main/java/com/auramart/catalog/web/GlobalExceptionHandler.java · replace

```
package com.auramart.catalog.web;

import com.auramart.catalog.product.ProductExceptions.DuplicateSkuException;
import com.auramart.catalog.product.ProductExceptions.ProductNotFoundException;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

import java.net.URI;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Turns exceptions into RFC 9457 "problem+json" responses.
 * Extending ResponseEntityExceptionHandler means Spring's own errors (malformed JSON,
 * a path id that isn't a UUID, unsupported methods...) get the same format for free.
 */
@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    private static final String TYPE_BASE = "https://auramart.dev/problems/";

    @ExceptionHandler(ProductNotFoundException.class)
    ProblemDetail handleNotFound(ProductNotFoundException ex) {
        return problem(HttpStatus.NOT_FOUND, "product-not-found", "Product not found",
                ex.getMessage());
    }

    @ExceptionHandler(DuplicateSkuException.class)
    ProblemDetail handleDuplicateSku(DuplicateSkuException ex) {
        return problem(HttpStatus.CONFLICT, "duplicate-sku", "Duplicate SKU", ex.getMessage());
    }

    /** Two requests changed the same product at the same time (@Version check failed). */
    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    ProblemDetail handleConcurrentUpdate(ObjectOptimisticLockingFailureException ex) {
        return problem(HttpStatus.CONFLICT, "concurrent-update", "Concurrent update",
                "The product was changed by someone else. Reload and try again.");
    }

    /** A database constraint caught what the service didn't, e.g. two creates racing. */
    @ExceptionHandler(DataIntegrityViolationException.class)
    ProblemDetail handleConstraintViolation(DataIntegrityViolationException ex) {
        return problem(HttpStatus.CONFLICT, "data-conflict", "Data conflict",
                "The request conflicts with existing data.");
    }
```

services/catalog-service/src/main/java/com/auramart/catalog/web/GlobalExceptionHandler.java · replace

```
    /** Validation failures: list every invalid field instead of one generic message. */
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(e -> errors.putIfAbsent(e.getField(), e.getDefaultMessage()));
        ProblemDetail body = ex.getBody();
        body.setType(URI.create(TYPE_BASE + "validation-error"));
        body.setTitle("Validation failed");
        body.setDetail("One or more fields are invalid.");
        body.setProperty("errors", errors);
        return handleExceptionInternal(ex, body, headers, status, request);
    }

    private static ProblemDetail problem(HttpStatus status, String type, String title,
                                          String detail) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(status, detail);
        problem.setType(URI.create(TYPE_BASE + type));
        problem.setTitle(title);
        return problem;
    }
}
```

Request samples

Create a `samples` folder in services/catalog-service with three JSON files, then replace `catalog.http`:

services/catalog-service/samples/new-product.json · new

```
{
  "sku": "KIT-MUG-350",
  "name": "Insulated Coffee Mug 350ml",
  "description": "Spill-proof steel mug with a sliding lid that keeps coffee hot for 6 hours.",
  "category": "kitchen",
  "brand": "Thermique",
  "price": 749.00,
  "currency": "INR",
  "stockQuantity": 50
}
```

services/catalog-service/samples/invalid-product.json · new

```
{
  "sku": "bad sku",
  "name": "",
  "price": -5,
  "currency": "RUPEES"
}
```

services/catalog-service/samples/update-flask.json · new

```
{
  "sku": "IGNORED-ON-UPDATE",
  "name": "Steel Vacuum Flask 750ml",
  "description": "Double-wall stainless steel flask that keeps tea or coffee hot for 18 hours.",
  "category": "kitchen",
  "brand": "Thermique",
  "price": 1199.00,
  "currency": "INR",
  "stockQuantity": 280
```

services/catalog-service/samples/update-flask.json · new

```
}
```

services/catalog-service/catalog.http · replace

```
# AuraMart Catalog API - run these with VS Code (REST Client extension) or IntelliJ.
# Click "Send Request" above each block. Ids below are from the seed data (V2).
@base = http://localhost:8081/api/v1/products

### 1. List products (paged, sorted by name)
GET {{base}}

### 2. Filter by category, cheapest first, 5 per page
GET {{base}}?category=kitchen&sort=price,asc&size=5

### 3. Ask for 500 per page -> capped at 100 (look at page.size in the response)
GET {{base}}?size=500

### 4. One product: the steel flask
GET {{base}}/0b9a7c1e-0003-4c3a-9a10-000000000003

### 5. Out of stock: the power bank -> "inStock": false
GET {{base}}/0b9a7c1e-0012-4c3a-9a10-000000000012

### 6. Unknown id -> 404 problem+json
GET {{base}}/00000000-0000-0000-0000-000000000000

### 7. Not a UUID -> 400
GET {{base}}/not-a-uuid

### 8. Create -> 201 + Location header
POST {{base}}
Content-Type: application/json

< ./samples/new-product.json

### 9. Same SKU again -> 409 duplicate-sku
POST {{base}}
Content-Type: application/json

< ./samples/new-product.json

### 10. Invalid body -> 400 with an "errors" map
POST {{base}}
Content-Type: application/json

< ./samples/invalid-product.json

### 11. Update the flask's price and stock -> 200 (the SKU in the body is ignored)
PUT {{base}}/0b9a7c1e-0003-4c3a-9a10-000000000003
Content-Type: application/json

< ./samples/update-flask.json

### 12. Deactivate the Java book -> 204
DELETE {{base}}/0b9a7c1e-0011-4c3a-9a10-000000000011

### 13. Read the Java book again -> 404 (deactivated products are hidden)
GET {{base}}/0b9a7c1e-0011-4c3a-9a10-000000000011

### 14. Health (database included)
GET http://localhost:8081/actuator/health
```

Check — restart the service, then try every case

Easiest: open catalog.http in VS Code and click **Send Request** on requests 8 to 13, in order. Or from PowerShell:

```
# Create -> HTTP/1.1 201, plus a Location header with the new id
curl.exe -i -X POST http://localhost:8081/api/v1/products `
    -H "Content-Type: application/json" `
    --data "@services/catalog-service/samples/new-product.json"

# Run the same command again -> HTTP/1.1 409 with "type": ".../duplicate-sku"

# Invalid body -> HTTP/1.1 400 with an "errors" map listing sku, name, price, currency...
curl.exe -i -X POST http://localhost:8081/api/v1/products `
    -H "Content-Type: application/json" `
    --data "@services/catalog-service/samples/invalid-product.json"

# Update the flask -> 200, price 1199.00, and the sku is still KIT-FLK-750
curl.exe -i -X PUT http://localhost:8081/api/v1/products/0b9a7c1e-0003-4c3a-9a10-000000000003 `
    -H "Content-Type: application/json" `
    --data "@services/catalog-service/samples/update-flask.json"

# Deactivate the Java book -> 204; reading it afterwards -> 404
curl.exe -i -X DELETE http://localhost:8081/api/v1/products/0b9a7c1e-0011-4c3a-9a10-000000000011
curl.exe -i http://localhost:8081/api/v1/products/0b9a7c1e-0011-4c3a-9a10-000000000011
```

Always type `curl.exe` in PowerShell (plain curl is a different command there), and send JSON from files: JSON typed inside PowerShell quotes breaks in Windows PowerShell 5.1.

Ship it

Branch	`feature/AM-7-product-crud`
Commit	`feat(catalog): product CRUD with problem+json errors (AM-7)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 13 · Browse with paging and sorting

AM-8 · 3 points · about 3 hours

> ### Goal
>
> Shoppers can filter by category, sort and page through products; nobody can request more than 100 at once.

Start — move the card to In progress

```
git switch main
git pull
git switch -c feature/AM-8-browse
```

The controller already accepts `category`, `page`, `size` and `sort` (see list() in ProductController). This story adds the page-size cap — four new lines under `spring:` in application.yml:

```
  data:
    web:
      pageable:
        max-page-size: 100   # nobody can ask for a million rows in one request
```

The whole file after the change:

services/catalog-service/src/main/resources/application.yml · replace

```
server:
```

services/catalog-service/src/main/resources/application.yml · replace

```
  port: 8081
  shutdown: graceful

spring:
  application:
    name: catalog-service

  # Java 21 virtual threads: each request gets a cheap thread, so blocking JDBC calls
  # no longer cap throughput the way a fixed 200-thread pool did.
  threads:
    virtual:
      enabled: true

  datasource:
    # Defaults point at docker-compose; override with environment variables elsewhere.
    url: ${CATALOG_DB_URL:jdbc:postgresql://localhost:5432/catalog_db}
    username: ${CATALOG_DB_USER:catalog_svc}
    password: ${CATALOG_DB_PASSWORD:catalog_pw}
    hikari:
      maximum-pool-size: 10
      pool-name: catalog-pool

  jpa:
    # Flyway owns the schema. Hibernate only checks that the entities match it.
    hibernate:
      ddl-auto: validate
    # Never hold a database connection open while the HTTP response is written.
    open-in-view: false
    properties:
      hibernate:
        jdbc:
          time_zone: UTC

  flyway:
    enabled: true
    locations: classpath:db/migration

  data:
    web:
      pageable:
        max-page-size: 100   # nobody can ask for a million rows in one request

  mvc:
    # RFC 9457 "problem+json" bodies for every error, including validation errors.
    problemdetails:
      enabled: true

management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics
  endpoint:
    health:
      probes:
        enabled: true          # /actuator/health/liveness and /readiness (Kubernetes later)
      show-details: always     # fine locally; restrict it in a real deployment

springdoc:
  swagger-ui:
    path: /swagger-ui.html

logging:
  level:
```

services/catalog-service/src/main/resources/application.yml · replace

```
    com.auramart: DEBUG
```

Check — restart the service, then:

```
curl.exe "http://localhost:8081/api/v1/products?category=kitchen&sort=price,asc&size=5"
curl.exe "http://localhost:8081/api/v1/products?size=500"
```

The response always has the same shape. With size=500 you get page.size 100:

```
{
  "content": [ { "id": "...", "sku": "...", "name": "...", "inStock": true, ... } ],
  "page": { "size": 100, "number": 0, "totalElements": 12, "totalPages": 1 }
}
```

- ☐ Kitchen products come back cheapest first.
- ☐ The deactivated Java book never appears in any list.
- ☐ size=500 is answered with page.size 100.

Ship it

Branch	`feature/AM-8-browse`
Commit	`feat(catalog): cap page size at 100 (AM-8)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 14 · Price and stock status

AM-9 · 2 points · about 2 hours

> **Goal**
>
> A product shows its price, currency, stock quantity and an inStock flag; unknown ids give 404.

Start — move the card to In progress

```
git switch main
git pull
git switch -c feature/AM-9-stock-status
```

The code is already in place — ProductResponse works out inStock from the entity. This story proves it on the edge cases; the tests in Step 15 then lock the behaviour in.

```
curl.exe http://localhost:8081/api/v1/products/0b9a7c1e-0012-4c3a-9a10-000000000012
#   power bank: "stockQuantity": 0, "inStock": false
curl.exe -i http://localhost:8081/api/v1/products/00000000-0000-0000-0000-000000000000
#   HTTP/1.1 404, "title": "Product not found"
curl.exe -i http://localhost:8081/api/v1/products/not-a-uuid
#   HTTP/1.1 400
```

Write what you saw as a short note in DEVLOG.md — that's the commit for this story.

Ship it

Branch	`feature/AM-9-stock-status`
Commit	`docs(catalog): verify stock status and 404 cases (AM-9)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 15 · Automated tests

AM-10 · 3 points · about 5 hours

> **Goal**
>
> 13 fast tests cover the service rules and the HTTP contract, and run on every push.

Start — move the card to In progress

```
git switch main
git pull
git switch -c feature/AM-10-tests
```

Spring Boot 4 splits test support by technology, so the web-layer test needs one extra test dependency: **spring-boot-starter-webmvc-test**. Replace the service's pom.xml:

services/catalog-service/pom.xml · replace

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>com.auramart</groupId>
        <artifactId>auramart-parent</artifactId>
        <version>0.1.0-SNAPSHOT</version>
        <relativePath>../../pom.xml</relativePath>
    </parent>

    <artifactId>catalog-service</artifactId>
    <name>AuraMart :: Catalog Service</name>
    <description>Product catalogue, stock levels and (Phase 4) semantic search</description>

    <dependencies>
        <!-- Boot 4 renamed spring-boot-starter-web to spring-boot-starter-webmvc -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <!-- Schema migrations. In Boot 4 the starter is needed for Flyway auto-configuration. -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-flyway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-database-postgresql</artifactId>
        </dependency>
        <dependency>
```

services/catalog-service/pom.xml · replace

```
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Swagger UI at /swagger-ui.html -->
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        </dependency>

        <!-- Tests: JUnit, AssertJ, Mockito -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <!-- Boot 4 splits test support by technology: this one brings @WebMvcTest + MockMvc -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Unit tests for the service — replace

No Spring and no database: Mockito stands in for the repository, so each test runs in milliseconds.

services/catalog-service/src/test/java/com/auramart/catalog/product/ProductServiceTest.java · replace

```
package com.auramart.catalog.product;

import com.auramart.catalog.product.ProductExceptions.DuplicateSkuException;
import com.auramart.catalog.product.ProductExceptions.ProductNotFoundException;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.util.Optional;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

/**
 * Fast unit tests: no Spring context, no database - each test runs in milliseconds.
```

services/catalog-service/src/test/java/com/auramart/catalog/product/ProductServiceTest.java · replace

```
 * Sprint 2 adds integration tests against a real PostgreSQL in a container.
 */
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    ProductRepository repository;

    @InjectMocks
    ProductService service;

    private static ProductRequest request(String sku) {
        return new ProductRequest(sku, "Steel Flask", "Keeps tea hot", "Kitchen", "Thermique",
                new BigDecimal("1299.00"), "inr", 10);
    }

    private static Product existingProduct() {
        return new Product("KIT-1", "Old name", "Old description", "kitchen", null,
                BigDecimal.TEN, "INR", 5);
    }

    @Test
    void create_normalisesCategoryAndCurrency() {
        when(repository.existsBySku("KIT-1")).thenReturn(false);
        when(repository.save(any(Product.class))).thenAnswer(call -> call.getArgument(0));

        ProductResponse created = service.create(request("KIT-1"));

        assertThat(created.category()).isEqualTo("kitchen");
        assertThat(created.currency()).isEqualTo("INR");
        assertThat(created.inStock()).isTrue();
    }

    @Test
    void create_rejectsDuplicateSku() {
        when(repository.existsBySku("KIT-1")).thenReturn(true);

        assertThatThrownBy(() -> service.create(request("KIT-1")))
                .isInstanceOf(DuplicateSkuException.class);
        verify(repository, never()).save(any());
    }

    @Test
    void get_throwsWhenMissingOrInactive() {
        UUID id = UUID.randomUUID();
        when(repository.findByIdAndActiveTrue(id)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> service.get(id))
                .isInstanceOf(ProductNotFoundException.class)
                .hasMessageContaining(id.toString());
    }

    @Test
    void update_changesDetailsButNeverTheSku() {
        UUID id = UUID.randomUUID();
        Product existing = existingProduct();
        when(repository.findByIdAndActiveTrue(id)).thenReturn(Optional.of(existing));
        when(repository.saveAndFlush(existing)).thenReturn(existing);

        ProductResponse updated = service.update(id, request("SOME-OTHER-SKU"));

        assertThat(updated.sku()).isEqualTo("KIT-1");
        assertThat(updated.name()).isEqualTo("Steel Flask");
        assertThat(updated.stockQuantity()).isEqualTo(10);
    }
```

services/catalog-service/src/test/java/com/auramart/catalog/product/ProductServiceTest.java · replace

```
    @Test
    void deactivate_hidesTheProduct() {
        UUID id = UUID.randomUUID();
        Product existing = existingProduct();
        when(repository.findByIdAndActiveTrue(id)).thenReturn(Optional.of(existing));

        service.deactivate(id);

        assertThat(existing.isActive()).isFalse();
    }

    @Test
    void zeroStock_meansNotInStock() {
        Product product = existingProduct();
        product.setStock(0);

        assertThat(product.isInStock()).isFalse();
    }

    @Test
    void negativeStock_isRejectedByTheEntity() {
        Product product = existingProduct();

        assertThatThrownBy(() -> product.setStock(-1))
                .isInstanceOf(IllegalArgumentException.class);
    }
}
```

Web-layer tests for the controller — new

@WebMvcTest starts only Spring MVC — the controller and the exception handler — with the service replaced by a mock (@MockitoBean replaces Boot 3's @MockBean). It checks status codes and JSON, not business rules.

services/catalog-service/src/test/java/com/auramart/catalog/product/ProductControllerTest.java · new

```
package com.auramart.catalog.product;

import com.auramart.catalog.product.ProductExceptions.DuplicateSkuException;
import com.auramart.catalog.product.ProductExceptions.ProductNotFoundException;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

import static org.hamcrest.Matchers.containsString;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.BDDMockito.given;
import static org.mockito.Mockito.verifyNoInteractions;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
```

services/catalog-service/src/test/java/com/auramart/catalog/product/ProductControllerTest.java · new

```
 * Web-layer slice test: starts only Spring MVC (controller + exception handler), with the
 * service replaced by a Mockito mock. Checks status codes and JSON, not business rules.
 */
@WebMvcTest(ProductController.class)
class ProductControllerTest {

    private static final String VALID_BODY = """
            {
              "sku": "KIT-MUG-350",
              "name": "Insulated Coffee Mug 350ml",
              "description": "Keeps coffee hot for 6 hours.",
              "category": "kitchen",
              "brand": "Thermique",
              "price": 749.00,
              "currency": "INR",
              "stockQuantity": 50
            }
            """;

    @Autowired
    MockMvc mvc;

    @MockitoBean
    ProductService service;

    private static ProductResponse mug(UUID id) {
        return new ProductResponse(id, "KIT-MUG-350", "Insulated Coffee Mug 350ml",
                "Keeps coffee hot for 6 hours.", "kitchen", "Thermique",
                new BigDecimal("749.00"), "INR", 50, true, Instant.parse("2026-10-26T10:00:00Z"));
    }

    @Test
    void create_returns201WithLocationHeader() throws Exception {
        UUID id = UUID.randomUUID();
        given(service.create(any(ProductRequest.class))).willReturn(mug(id));

        mvc.perform(post("/api/v1/products")
                        .contentType(MediaType.APPLICATION_JSON).content(VALID_BODY))
                .andExpect(status().isCreated())
                .andExpect(header().string("Location", containsString("/api/v1/products/" + id)))
                .andExpect(jsonPath("$.sku").value("KIT-MUG-350"))
                .andExpect(jsonPath("$.inStock").value(true));
    }

    @Test
    void create_withInvalidBody_returns400WithFieldErrors() throws Exception {
        String invalid = """
                { "sku": "bad sku", "name": "", "price": -5, "currency": "RUPEES" }
                """;

        mvc.perform(post("/api/v1/products")
                        .contentType(MediaType.APPLICATION_JSON).content(invalid))
                .andExpect(status().isBadRequest())
                .andExpect(jsonPath("$.title").value("Validation failed"))
                .andExpect(jsonPath("$.errors.sku").exists())
                .andExpect(jsonPath("$.errors.price").exists())
                .andExpect(jsonPath("$.errors.currency").exists());
        verifyNoInteractions(service);
    }

    @Test
    void create_withDuplicateSku_returns409() throws Exception {
        given(service.create(any(ProductRequest.class)))
                .willThrow(new DuplicateSkuException("KIT-MUG-350"));
```

services/catalog-service/src/test/java/com/auramart/catalog/product/ProductControllerTest.java · new

```
        mvc.perform(post("/api/v1/products")
                        .contentType(MediaType.APPLICATION_JSON).content(VALID_BODY))
                .andExpect(status().isConflict())
                .andExpect(jsonPath("$.type")
                        .value("https://auramart.dev/problems/duplicate-sku"));
    }

    @Test
    void get_unknownProduct_returns404ProblemDetail() throws Exception {
        UUID id = UUID.randomUUID();
        given(service.get(id)).willThrow(new ProductNotFoundException(id));

        mvc.perform(get("/api/v1/products/{id}", id))
                .andExpect(status().isNotFound())
                .andExpect(jsonPath("$.status").value(404))
                .andExpect(jsonPath("$.detail").value(containsString(id.toString())));
    }

    @Test
    void get_withMalformedId_returns400() throws Exception {
        mvc.perform(get("/api/v1/products/{id}", "not-a-uuid"))
                .andExpect(status().isBadRequest());
    }

    @Test
    void delete_returns204() throws Exception {
        mvc.perform(delete("/api/v1/products/{id}", UUID.randomUUID()))
                .andExpect(status().isNoContent());
    }
}
```

Check

```
.\mvnw.cmd test
#   Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
#   BUILD SUCCESS

# Run one class while you work on it:
.\mvnw.cmd -pl services/catalog-service test "-Dtest=ProductControllerTest"
```

Break something on purpose — for example, make the duplicate-SKU handler return HttpStatus.OK instead of CONFLICT — and watch a test fail. Then undo it. That's how you know the tests protect you.

Ship it

Branch	`feature/AM-10-tests`
Commit	`test(catalog): service unit tests and web-layer tests (AM-10)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 16 · Swagger UI and health checks

AM-11 · 1 point · about 1 hour

> **Goal**
>
> Swagger UI lists every endpoint under a proper title, and the health endpoint includes the database.

Start — move the card to In progress

```
git switch main
git pull
git switch -c feature/AM-11-swagger
```

services/catalog-service/src/main/java/com/auramart/catalog/web/OpenApiConfig.java · new

```
package com.auramart.catalog.web;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/** Title, version and description shown at the top of Swagger UI (/swagger-ui.html). */
@Configuration
public class OpenApiConfig {

    @Bean
    OpenAPI catalogOpenApi() {
        return new OpenAPI().info(new Info()
                .title("AuraMart Catalog API")
                .version("v1")
                .description("Products and stock levels. Semantic search arrives in Phase 4."));
    }
}
```

Check — restart the service, then:

```
# In the browser: http://localhost:8081/swagger-ui.html
#   -> "AuraMart Catalog API v1" with 5 product endpoints
curl.exe http://localhost:8081/actuator/health
#   {"status":"UP","components":{"db":{"status":"UP",...},...}}
```

Try a request from Swagger UI itself: expand GET /api/v1/products → Try it out → Execute.

Ship it

Branch	`feature/AM-11-swagger`
Commit	`feat(catalog): OpenAPI title and version (AM-11)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Step 17 · Documentation that works on a fresh clone

AM-12 · 1 point · about 1.5 hours

> **Goal**
>
> Decisions are indexed, the user stories are written down, and the README's instructions work exactly as written on a brand-new clone.

Start — move the card to In progress

```
git switch main
git pull
git switch -c docs/AM-12-docs
```

docs/adr/README.md · new

```
# Architecture Decision Records

One short file per important decision: the context, the decision and its consequences.

| ADR | Decision | Status |
|---|---|---|
| [0001][adr1] | Spring Boot 4.1 instead of 3.x | Accepted |
| [0002][adr2] | No Eureka: platform DNS for discovery | Accepted |
| [0003][adr3] | One Postgres, one database per service | Accepted |

New ADR: copy the newest file, give it the next number, then add a row and a link here.

[adr1]: 0001-spring-boot-4-instead-of-3.md
[adr2]: 0002-no-eureka-service-discovery.md
[adr3]: 0003-one-postgres-database-per-service.md
```

Copy three more files from 4-phase1-complete: `docs/01-requirements/user-stories.md` (the Sprint 0 and Sprint 1 stories with their acceptance criteria), `services/catalog-service/README.md` and the main `README.md` — its run section now uses `.\mvnw.cmd` and it has a Project status checklist. Read each one before you commit it.

Ship it

Branch	`docs/AM-12-docs`
Commit	`docs: ADR index, user stories and verified run instructions (AM-12)`
Then	The story loop from Step 10: push, pull request, CI, squash merge.

Check — after the merge, the fresh-clone test

```
cd $env:TEMP
git clone https://github.com/<your-github-username>/auramart auramart-check
cd auramart-check
.\mvnw.cmd -B verify          # must pass with nothing but the README's instructions
cd ..
Remove-Item -Recurse -Force auramart-check
cd C:\dev\auramart
```

If anything in the README doesn't work exactly as written, fix it in a small follow-up pull request.

Part D — Finish Phase 1

Step 18 · Sprint review and release v0.1.0

Sat 7 Nov · about 45 minutes

> **Goal**
>
> You demo every story against its acceptance criteria and publish release v0.1.0 on GitHub.

Reset to a clean start, so the demo behaves exactly as described

```
git switch main
git pull
docker compose down -v
docker compose up -d
.\mvnw.cmd -pl services/catalog-service spring-boot:run
```

Demo — about 5 minutes; record your screen while you do it

1. docker compose ps — the infrastructure is healthy.
2. The Flyway log line and the 12 products (AM-6).
3. catalog.http requests 8 to 13: create 201, duplicate 409, invalid 400 with errors, update 200, deactivate 204, then 404 (AM-7).
4. Requests 2 and 3: category with sort, and the 100 cap (AM-8). Request 5: the power bank is out of stock (AM-9).
5. The Actions tab: 13 tests green on every merge (AM-10). Swagger UI and /actuator/health (AM-11).
6. The README and docs on GitHub (AM-12).

Release

```
git tag -a v0.1.0 -m "Sprint 1: Catalog API"
git push origin v0.1.0
gh release create v0.1.0 --title "v0.1.0 - Catalog API" --generate-notes
```

Check

- ☐ The repo's Releases page shows v0.1.0 with the merged pull requests listed.
- ☐ Every Sprint 1 issue is closed and every card is in Done.
- ☐ The demo video is saved (you will reuse it for LinkedIn and interviews).

Step 19 · Retrospective, velocity and what's next

Sat 7 Nov · about 30 minutes

Goal

You know your real velocity, and Sprint 2 starts with one concrete improvement.

```
### Sprint 1 retrospective - Sat 7 Nov 2026
Energy (1-5):
What went well:
What didn't:
What I learned:
Last sprint's action done? yes / no
One action for next sprint (small, specific):
Velocity: planned 17 points, completed __ points
```

Next

Monday 9 November starts Sprint 2: the API Gateway, CORS, request IDs and Testcontainers integration tests against a real PostgreSQL (Execution Plan, Sprint 2 page).

Phase 1 is complete when every box is ticked

- ☐ Main is protected, CI is required, and the badge is green.
- ☐ The infrastructure starts with one command and passes its health checks.
- ☐ All 7 Sprint 1 stories are merged through pull requests.
- ☐ 13 tests pass locally and in CI.
- ☐ Swagger UI, health, ADR index, user stories and DEVLOG are in place.
- ☐ v0.1.0 is released on GitHub.

Appendix A. The project at the end of Phase 1

```
auramart/
|-- .editorconfig  .gitattributes  .gitignore  .env.example
|-- .github/
|   |-- pull_request_template.md
|   `-- workflows/ci.yml
|-- CONTRIBUTING.md  DEVLOG.md  README.md
|-- docker-compose.yml
|-- pom.xml                                  parent POM
|-- mvnw  mvnw.cmd  .mvn/                    Maven wrapper (Step 2)
|-- docs/
|   |-- 01-requirements/user-stories.md
|   `-- adr/  README.md  0001-...  0002-...  0003-...
|-- infra/
|   |-- postgres/init/01-create-service-databases.sql
|   |-- windows/wslconfig.example
|   `-- k8s/metrics-server-patch.json
|-- scripts/  README.txt  install-tools.ps1  check-env.ps1  predownload.ps1
|             create-sprint1-issues.ps1
`-- services/catalog-service/
    |-- pom.xml  README.md  catalog.http
    |-- samples/  new-product.json  invalid-product.json  update-flask.json
    `-- src/
        |-- main/java/com/auramart/catalog/
        |   |-- CatalogServiceApplication.java
        |   |-- product/  Product  ProductController  ProductExceptions
        |   |             ProductRepository  ProductRequest  ProductResponse  ProductService
        |   `-- web/      GlobalExceptionHandler  OpenApiConfig
        |-- main/resources/
        |   |-- application.yml
        |   `-- db/migration/  V1__create_products.sql  V2__seed_demo_products.sql
        `-- test/java/com/auramart/catalog/product/
            ProductServiceTest  ProductControllerTest
```

What changes between 3-phase1-starter and 4-phase1-complete

File	Change	Step
docker-compose.yml, .gitattributes, .editorconfig, .github/, infra/k8s, infra/windows, scripts/ (setup)	from the kit	2
README.md	CI badge; later run steps and project status	6, 17
CONTRIBUTING.md, DEVLOG.md, scripts/create-sprint1-issues.ps1	new	6, 7
Product.java, GlobalExceptionHandler.java, catalog.http · samples/*.json	replaced · new	12
application.yml	replaced — page-size cap	13
catalog-service pom.xml, ProductServiceTest.java · ProductControllerTest.java	replaced · new	15
OpenApiConfig.java	new	16
docs/adr/README.md, docs/01-requirements/user-stories.md, catalog-service README.md	new	17
Everything else: V1, V2, records, repository, service, controller, parent POM	unchanged	—

Appendix B. Build files

pom.xml · from the starter

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  AuraMart - parent POM (aggregator + shared dependency management).

  Every service inherits Spring Boot's managed versions from here, and the Spring Cloud
  and Spring AI BOMs are imported once, so no child POM ever pins a Spring version.
  Upgrading the platform = editing this one file.
-->
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <!-- Use the newest 4.1.x patch: https://spring.io/projects/spring-boot -->
        <version>4.1.1</version>
        <relativePath/>
    </parent>

    <groupId>com.auramart</groupId>
    <artifactId>auramart-parent</artifactId>
    <version>0.1.0-SNAPSHOT</version>
    <packaging>pom</packaging>
    <name>AuraMart</name>
    <description>Event-driven e-commerce platform with an AI shopping assistant</description>

    <modules>
        <!-- Phase 1 -->
        <module>services/catalog-service</module>
        <!-- Phase 2: <module>services/api-gateway</module> -->
        <!-- Phase 3: <module>libs/event-contracts</module>
                      <module>services/order-service</module>
                      <module>services/notification-service</module> -->
        <!-- Phase 4: <module>services/ai-assistant-service</module> -->
    </modules>

    <properties>
        <java.version>21</java.version>
        <spring-cloud.version>2025.1.2</spring-cloud.version>
        <spring-ai.version>2.0.1</spring-ai.version>
        <springdoc.version>3.1.0</springdoc.version>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>${spring-cloud.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
            <dependency>
                <groupId>org.springframework.ai</groupId>
                <artifactId>spring-ai-bom</artifactId>
                <version>${spring-ai.version}</version>
                <type>pom</type>
```

pom.xml · from the starter

```
                <scope>import</scope>
            </dependency>
            <dependency>
                <groupId>org.springdoc</groupId>
                <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
                <version>${springdoc.version}</version>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <build>
        <pluginManagement>
            <plugins>
                <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
                </plugin>
            </plugins>
        </pluginManagement>
    </build>
</project>
```

services/catalog-service/src/main/java/com/auramart/catalog/CatalogServiceApplication.java · from the starter

```
package com.auramart.catalog;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CatalogServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(CatalogServiceApplication.class, args);
    }
}
```

Appendix C. docker-compose.yml

Phase 1's core services start by default; the tools for later sprints are opt-in profiles.

docker-compose.yml · from the kit

```
# ================================================================
# AuraMart - local infrastructure
#
#   docker compose up -d                          Phase 1 core: Postgres, Redis, Kafka, Kafka UI
#   docker compose --profile mail up -d           + Mailpit            (Sprint 6)
#   docker compose --profile observability up -d  + Grafana LGTM stack (Sprint 11)
#   docker compose --profile security up -d       + Keycloak           (Sprint 12)
#   docker compose --profile ai up -d             + Ollama in Docker   (no native Ollama)
#   docker compose stop                           stop, keep data
#   docker compose down -v                        stop and WIPE all data (re-runs the DB init)
#
# The Spring Boot services are not in here: run them from the IDE until Phase 6.
# ================================================================

name: auramart

services:

  # PostgreSQL 17 + pgvector - one database and login per service (infra/postgres/init)
  postgres:
    image: pgvector/pgvector:pg17
    container_name: auramart-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_ADMIN_USER:-auramart_admin}
      POSTGRES_PASSWORD: ${POSTGRES_ADMIN_PASSWORD:-admin_pw}
      POSTGRES_DB: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./infra/postgres/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U ${POSTGRES_ADMIN_USER:-auramart_admin} -d postgres
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped

  # Redis - product cache from Sprint 3
  redis:
    image: redis:7.4-alpine
    container_name: auramart-redis
    command:
      - redis-server
      - --appendonly
      - "yes"
      - --maxmemory
      - 256mb
      - --maxmemory-policy
      - allkeys-lru
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
```

docker-compose.yml · from the kit

```
      timeout: 3s
      retries: 10
    restart: unless-stopped

  # Apache Kafka 4 - KRaft mode, one node acting as broker and controller
  #   EXTERNAL  localhost:9092  -> your Spring apps running on Windows
  #   PLAINTEXT kafka:19092     -> other containers (Kafka UI)
  kafka:
    image: apache/kafka:4.1.2
    container_name: auramart-kafka
    ports:
      - "9092:9092"
    environment:
      CLUSTER_ID: "4L6g3nShT-eMCtK--X86sw"
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: >-
        CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"   # topics are declared in code
      KAFKA_NUM_PARTITIONS: 3
      KAFKA_LOG_DIRS: /var/lib/kafka/data
      KAFKA_HEAP_OPTS: "-Xms512m -Xmx512m"       # keep Kafka laptop-sized
    volumes:
      - kafka-data:/var/lib/kafka/data
    healthcheck:
      test:
        - CMD-SHELL
        - >-
          /opt/kafka/bin/kafka-broker-api-versions.sh
          --bootstrap-server localhost:9092 > /dev/null 2>&1
      interval: 10s
      timeout: 10s
      retries: 12
      start_period: 15s
    restart: unless-stopped

  # Kafka UI - http://localhost:8090
  kafka-ui:
    image: ghcr.io/kafbat/kafka-ui:latest
    container_name: auramart-kafka-ui
    ports:
      - "8090:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: auramart-local
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:19092
      DYNAMIC_CONFIG_ENABLED: "true"
    depends_on:
      kafka:
        condition: service_healthy
    restart: unless-stopped

  # ---------------------------------------------------------------- opt-in profiles

  # Mailpit (Sprint 6) - catches emails locally. SMTP localhost:1025, inbox localhost:8025
  mailpit:
    image: axllent/mailpit:latest
```

docker-compose.yml · from the kit

```
    container_name: auramart-mailpit
    profiles: ["mail"]
    ports:
      - "1025:1025"
      - "8025:8025"
    restart: unless-stopped

  # Grafana LGTM (Sprint 11) - OpenTelemetry collector, Prometheus, Tempo, Loki, Grafana
  #   Grafana http://localhost:3000 - OTLP gRPC localhost:4317 - OTLP HTTP localhost:4318
  lgtm:
    image: grafana/otel-lgtm:latest
    container_name: auramart-lgtm
    profiles: ["observability"]
    ports:
      - "3000:3000"
      - "4317:4317"
      - "4318:4318"
    restart: unless-stopped

  # Keycloak (Sprint 12) - admin console http://localhost:8180 (8080 is the API Gateway)
  keycloak:
    image: quay.io/keycloak/keycloak:26.8.1
    container_name: auramart-keycloak
    profiles: ["security"]
    command: ["start-dev"]
    environment:
      KC_BOOTSTRAP_ADMIN_USERNAME: ${KEYCLOAK_ADMIN_USER:-admin}
      KC_BOOTSTRAP_ADMIN_PASSWORD: ${KEYCLOAK_ADMIN_PASSWORD:-admin}
    ports:
      - "8180:8080"
    restart: unless-stopped

  # Ollama in Docker - only if you do NOT run the native app (both use port 11434).
  # On Windows the native app is faster because it can use your GPU.
  ollama:
    image: ollama/ollama:latest
    container_name: auramart-ollama
    profiles: ["ai"]
    ports:
      - "11434:11434"
    volumes:
      - ollama-data:/root/.ollama
    restart: unless-stopped

volumes:
  postgres-data:
  redis-data:
  kafka-data:
  ollama-data:
```

Appendix D. Troubleshooting Phase 1

Problem	Likely cause	Fix
“Cannot find path …\Downloads\AuraMart\2-laptop-setup”	The folder is somewhere else, or one level too deep (File Explorer makes AuraMart\AuraMart)	Move the AuraMart folder that holds START-HERE.txt into Downloads, or unzip again with the command in Step 1.
“Validate failed: Migration checksum mismatch”	A migration file changed after it ran	Undo the edit and put the change in a new V3 file. Local data only: docker compose down -v, then up -d.
“Schema-validation: wrong column type encountered in column …”	An entity field doesn't match the Flyway table	Compare the field with V1. Product.java must use columnDefinition = "timestamptz" for the two timestamps.
“Schema-validation: missing table [products]”	Flyway didn't run	Check that spring-boot-starter-flyway and flyway-database-postgresql are in the catalog pom.xml.
“Connection to localhost:5432 refused”	Infrastructure not running	Start Docker Desktop, then docker compose up -d.
“password authentication failed for user catalog_svc”	The data volume existed before the init script	docker compose down -v, then up -d.
“Port 8081 was already in use”	Another copy of the service is running	Stop it (IntelliJ's stop button, or Ctrl+C in its terminal).
“package org.springframework.boot.webmvc.test.autoconfigure does not exist”	Missing Boot 4 test starter	Add spring-boot-starter-webmvc-test (test scope) to the catalog pom.xml (Step 15).
HTTP 415 Unsupported Media Type	No Content-Type header	Add -H "Content-Type: application/json".
HTTP 400 for a body that looks right	PowerShell mangled the JSON	Send it from a file: --data "@path\to\file.json".
“mvnw is not recognized”	Windows needs the .cmd file	Use .\mvnw.cmd (./mvnw on macOS and Linux).
CI: “./mvnw: Permission denied” or “sh\r”	mvnw lost its execute bit or has Windows line endings	git update-index --chmod=+x mvnw ; git add --renormalize . ; commit and push.
gh pr merge refuses: “base branch policy prohibits the merge”	The build check hasn't passed	gh pr checks --watch, fix any failure, push again.
git push to main is rejected	The ruleset is working	Use a branch and a pull request (Step 10).